Write Strategies

Write strategies are caching techniques optimized to reduce **cache staleness** during write operations. They usually trade speed, complexity, or cost for fresher data.

Main strategies:

```
None
Write-Through

Write-Back / Write-Behind

Write-Around
```

1. Why Write Strategies Matter

If data changes in database but cache is old:

```
None
User sees stale data

Wrong balances

Old product stock

Incorrect dashboard numbers
```

Write strategies keep cache synchronized.

2. Write-Through

Every write goes through cache first, then database.

```
None
Application
   ↓
Cache
   ↓
Database
```

3. Write-Through Flow

```
None
Update Product Price
   ↓
Write cache
   ↓
Write DB
   ↓
Success
```

4. Example

```
None
price = ₹500 → ₹450
```

Both updated immediately:

```
None
Redis = ₹450

MySQL = ₹450
```

5. Advantages of Write-Through

```
None
Cache always fresh

Strong consistency between cache and DB

Reads become fast immediately

Simple data correctness model
```

6. Disadvantages of Write-Through

```
None
Slower writes (2 writes)

Higher infrastructure cost

Most written data may never be read

Cold start on new cache node

Need eviction policy if cache smaller than DB
```

7. Cold Start Problem

New cache node starts empty:

```
None
Many cache misses initially
```

Fix using:

```
None
Cache-aside

Warmup scripts

Preload hot keys
```

8. Write-Back / Write-Behind

Application writes only to cache first.

Database updated later asynchronously.

```
App
 ↓
Cache
 ↓ (later flush)
Database
```

9. Write-Back Flow

```
User likes post
→ Increment counter in Redis
→ Return success instantly
→ Flush to DB every minute
```

10. Advantages of Write-Back

```
None
```

```
Very fast writes

Low DB write pressure

Batch updates possible

Good for burst traffic
```

11. Disadvantages of Write-Back

```
None
```

```
Data loss risk if cache crashes

More complex architecture

Need HA cache cluster

Temporary inconsistency

Hard retry/recovery logic
```

12. Best Use Cases of Write-Back

Use for:

- Counters
- Analytics events
- Likes/views
- Metrics ingestion
- Temporary high-volume writes

13. Write-Through vs Write-Back

Feature	Write-Through	Write-Back
Write Speed	Slower	Faster
Consistency	Stronger	Eventual
DB Load	Higher	Lower
Complexity	Lower	Higher
Risk of Data Loss	Low	Higher

14. Write-Around

Application writes directly to database.

Cache is skipped during write.

```
None
App

 ↓

Database
```

Later reads use cache-aside or read-through.

15. Write-Around Flow

```
None
Update Product Stock

→ Write DB only


Later read:

Cache miss → fetch DB → cache result
```

16. Advantages of Write-Around

None

No unnecessary cache pollution

Good when writes are frequent but reads rare

Lower cache memory usage

17. Disadvantages of Write-Around

None

Next read may be slower (cache miss)

Possible stale cache if old value not invalidated

Higher DB read after writes

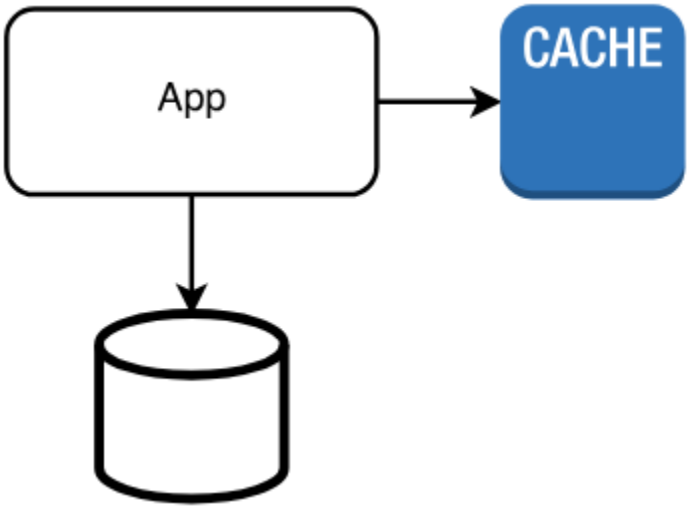
18. Best Use Cases of Write-Around

Use when:

- Many writes, few reads
- Logging systems
- Large datasets
- Rarely reused values

19. Architecture Options

Write-Around + Cache-Aside

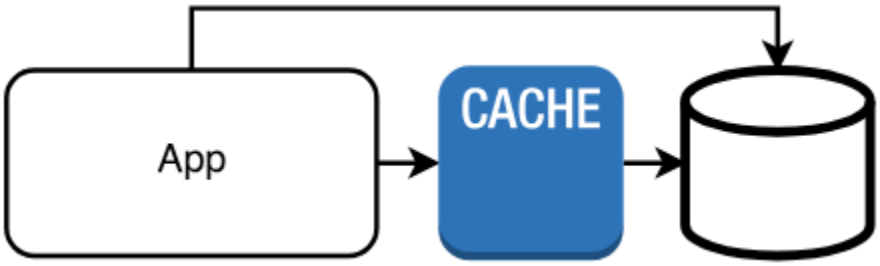



```
None

Write → DB

Read → Cache first, miss then DB
```

Write-Around + Read-Through




```
None

Write → DB

Read → Cache fetches DB automatically
```

20. HLD Recommendations

E-commerce

```
None

Write-through for inventory
```

Social Media Likes

```
None

Write-back counters
```

Logging

```
None
Write-around
```

21. Interview Answer

If asked “How to reduce stale cache?”

```
None
Use write-through for critical data

Use write-back for counters/events

Use write-around for write-heavy low-read workloads

Add TTL + invalidation
```

Strong answer.

22. Advantages Summary

```
None
Fresh cache

Better performance

Flexible workload optimization

Lower DB pressure (write-back)
```

23. Disadvantages Summary

```
None
```

```
Complexity

Latency tradeoffs

Possible stale reads

Possible data loss (write-back)
```

24. One-Line Summary

Write-through, write-back, and write-around are caching write strategies that balance consistency, speed, and cost depending on how critical freshness and write performance are.